

TEXNİKİ SƏNƏDLƏŞMƏ

Servislərin Detallı izahı

8 mikroservisin hər birinin məsuliyyəti, entity-ləri, endpoint-ləri,
Kafka integrasiyası və daxili qərarları

E-Sağlamlıq Sığortası — Mikroservis Backend Platforması
Java 17 · Spring Boot 3.2.5 · Kafka · PostgreSQL · 2026

Ümumi mənzərə

Sistem **8 ayrıca repozitoriyadan** ibarətdir: 1 paylaşılan kitabxana (infra), 1 gateway və 6 biznes servisi. Hər servis öz `build.gradle`, öz bazası və öz Docker konteyneri ilə müstəqildir. Aşağıdakı cədvəl xülasədir; sonrakı bölmələrdə hər biri ayrıca açıqlanır.

Servis	Port	Baza	Əsas məsuliyyət
infra	—	—	Paylaşılan enum, event, exception, DTO (jar)
gateway	8080	—	Marşrutlama + JWT doğrulama (giriş nöqtəsi)
iam	8081	ehi_iam	Qeydiyyat, login, JWT, istifadəçi idarəsi
policy	8082	ehi_policy	Planlar və police-lər (sığorta müqavilələri)
claim	8083	ehi_claim	İddialar, sübut faylları, baxış
payment	8084	ehi_payment	Mock ödəniş (premium + iddia ödənişi)
ai	8085	ehi_ai	Fırıldaq qiymətləndirmə, risk profili, chatbot
notification	8086	ehi_notification	Bütün hadisələrdən bildiriş yaratmaq

1. infra — Paylaşılan Kitabxana

Spring Boot deyil, sadə `java-library` jar-ıdır. Spring asılılığı yoxdur — yüngül qalsın deyə. Bütün servislərin paylaştığı tipləri saxlayır və `./gradlew publishToMavenLocal` ilə `mavenLocal()` -a yerləşdirilir.

Tərkibi

- **8 enum:** `UserRole`, `PolicyStatus`, `ClaimStatus`, `ClaimType`, `PaymentStatus`, `PaymentReferenceType`, `NotificationType`, `NotificationChannel`.
- **8 event DTO (record):** `UserRegistered`, `PolicyCreated`, `PaymentCompleted`, `PaymentFailed`, `ClaimSubmitted`, `ClaimDecision`, `FraudDetected`, `Notification`.
- **KafkaTopics** sabitləri — topic adları heç vaxt sətir kimi hardcode edilmir.
- **Cavab zərfləri:** `ApiResponse<T>`, `PagedResponse<T>`, `ErrorResponse`.
- **Exception modeli:** `BaseErrorService` interfeysi + `BaseErrorEnum` + `BaseException` və 4 hazır alt-sinif + `ServiceException`.
- **DateUtil:** bütün vaxtlar `Instant` ilə işlənir, `LocalDateTime` yox.

Niyə vacibdir: Bütün servislər eyni event və enum tiplərini import edir — heç bir tip dublikat olunmur. Bir hadisənin formatı dəyişəndə yalnız infra-da dəyişir.

2. gateway — API Gateway

Spring Cloud Gateway (reaktiv/WebFlux) üzərində qurulub. Bazası yoxdur — sırf marşrutlama və autentifikasiya çoxluğudur. `infra` -dan asılı deyil.

Necə işləyir

- **JwtAuthFilter** (`order = -1` , marşrutlamadan əvvəl): yolu public siyahıya görə yoxlayır.
- **Public yollar:** `/api/v1/auth/**` (hər metod) və `/api/v1/plans/**` (yalnız GET).
- Qalan yollar üçün `Authorization: Bearer <token>` tələb olunur; yoxdursa/yanlıqdırsa → **401**.
- Doğru token-də sorğuya `X-User-Id` və `X-User-Role` başlıqları əlavə olunur.

Marşrut prefiksi	Hədəf servis	Public?
/api/v1/auth/**	iam :8081	Bəli
/api/v1/users/**	iam :8081	Xeyr
/api/v1/plans/** (GET)	policy :8082	Bəli
/api/v1/policies/**	policy :8082	Xeyr
/api/v1/claims/**	claim :8083	Xeyr
/api/v1/payments/**	payment :8084	Xeyr
/api/v1/ai/**	ai :8085	Xeyr
/api/v1/notifications/**	notification :8086	Xeyr

3. iam — Kimlik və Giriş İdarəsi

Yeganə servisdır ki, parolları idarə edir və JWT token-ləri verir. Digər servislər token-i yalnız doğrular. IAM-da JWT subyekti **email**-dir (digər servislərdə userId).

Entity: User (users)

Sahə	Tip	Qeyd
id	UUID	PK
email	String	unikal
password	String	BCrypt hash
firstName / lastName	String	—
role	UserRole	ADMIN / STAFF / CUSTOMER
active	Boolean	false = bloklanmış (login qadağan)
createdAt / updatedAt	Instant	—

Əsas endpoint-lər

Metod	Yol	Auth
POST	/api/v1/auth/register	Public
POST	/api/v1/auth/login	Public
POST	/api/v1/auth/refresh	Bearer
GET / PUT	/api/v1/users/me	Bearer
POST	/api/v1/users/me/password	Bearer
PATCH	/api/v1/users/{id}/role	ADMIN
PATCH	/api/v1/users/{id}/status	ADMIN
GET	/api/v1/users/search?query=	ADMIN, STAFF

Kafka: qeydiyyatdan sonra `user.registered` hadisəsini yayımlayır (consumer yoxdur). Login zamanı `active=false` olan istifadəçi rədd olunur (*Account is suspended*).

4. policy — Planlar və Police-lər

İki entity: **Plan** (sığorta məhsulu kataloqu) və **Policy** (müşərinin aldığı konkret müqavilə). Plan qiyməti alış anında police-yə kopyalanır ki, sonrakı qiymət dəyişikliyi köhnə police-lərə təsir etməsin.

Plan
name, description, coverageAmount,
premiumAmount, durationMonths, active

Policy
policyNumber (POL-xxxx), userId, plan
(ManyToOne), status, premiumAmount, startDate,
endDate

Police həyat dövrü

Alış → **PENDING** (startDate/endDate boş) → ödəniş tamamlananda → **ACTIVE** (endDate = start + durationMonths). Ləğv yalnız PENDING/ACTIVE-dən mümkündür → CANCELLED.

Metod	Yol	Auth
GET	/api/v1/plans · /plans/{id}	Public
POST	/api/v1/plans	ADMIN
POST	/api/v1/policies	CUSTOMER
GET	/api/v1/policies/me	CUSTOMER
PUT	/api/v1/policies/{id}/cancel	CUSTOMER, ADMIN

Kafka: alışda `policy.created` yayımlayır; `payment.completed` (yalnız POLICY_PREMIUM) qəbul edib police-ni aktivləşdirir. Aktivləşdirmə `@Transactional` -dir (lazy Plan proxy üçün) və idempotentdir.

5. claim — İddialar

Sistemin mərkəzi. Müştəri iddia göndərir, AI avtomatik qiymətləndirir, lazım gələrsə əməkdaş yoxlayır. Sübut faylları (multipart) diskdə saxlanılır.

Entity: Claim (`claims`)

claimNumber (CLM-xxxx), userId, policyId, claimType, amount, description, status, approvedAmount, rejectionReason, reviewedBy, **riskScore**, **fraudFlags**, **aiExplanation**. Ayrıca `ClaimEvidence` (fayl metadatası).

AI avtomatik qərar (applyFraudResult)

`fraud.detected` qəbul edildikdə risk balına görə avtomatik qərar verilir. Status `SUBMITTED` deyilsə heç nə edilmir (manual baxış üzərinə yazılır):

Risk balı	Avtomatik qərar	Nəticə
< 40	APPROVED	Tam məbləğ təsdiqlənir, claim.decision yayımlanır
40–69	UNDER_REVIEW	Əməkdaş növbəsinə düşür (event yox)
≥ 70	REJECTED	Rədd edilir, claim.decision yayımlanır

Manual baxış (reviewClaim): yalnız STAFF/ADMIN, yalnız SUBMITTED/UNDER_REVIEW statusda. APPROVED üçün `0 < approvedAmount ≤ amount` tələb olunur.

Kafka: göndərişdə `claim.submitted` ; qərar (manual və ya AI) `claim.decision` ; `fraud.detected` qəbul edir.

6. payment — Ödəniş (Mock)

MVP üçün ödəniş **simulyasiyadır**: `MockPaymentProcessor` 2 saniyə gecikir, sonra avtomatik COMPLETED edir. İki istiqamətdə işləyir — premium ödənişi (police üçün) və iddia ödənişi (müşəriyə).

Entity: Payment

userId, referenceld (policyId və ya claimId), **referenceType** (POLICY_PREMIUM / CLAIM_PAYOUT), amount, status, transactionId (TXN-xxxx), failureReason.

İdempotentlik (kritik düzəliş): `processPayment` əvvəlcə eyni `(referenceId, referenceType)` üçün FAILED olmayan ödəniş axtarır — varsa, yenisini yaratmadan mövcudu qaytarır. Bu, Kafka təkrar çatdırılması zamanı *ikiqat ödənişin* qarşısını alır.

Kafka: `policy.created` → POLICY_PREMIUM ödənişi; `claim.decision` (yalnız APPROVED) → CLAIM_PAYOUT; nəticədə `payment.completed` və ya `payment.failed` yayımlanır.

7. ai — Fırıldağ, Risk və Chatbot

Üç vəzifəni daşıyır. `claim.submitted` qəbul edib hibrid fıırıldağ qiymətləndirməsi aparır və `fraud.detected` yayımlayır.

Hibrid fıırıldağ aşkarlanması

- Qayda mühərriki (rule engine):** claimType üzrə məbləğ hədləri (HOSPITALIZATION 10000, MEDICATION 2000, DENTAL 1500, CONSULTATION 500) + təkrar yüksək-riskli istifadəçi yoxlaması → `ruleScore`.
- AI çağırışı:** yalnız `ruleScore ≥ 40` olduqda OpenRouter/Gemini 2.5 Flash çağırılır. Cavab markdown fenced-lərindən təmizlənilib JSON kimi parse olunur.
- Yekun bal:** AI işləyibsə `finalScore = ruleScore×0.4 + aiScore×0.6`, əks halda yalnız ruleScore.
- Fallback:** AI xətası/timeout olarsa (default 15s) sırf qaydaya əsaslanan bala düşür — sistem dayanmır.

Digər iki vəzifə

- RiskProfile:** hər istifadəçi üçün bir sətir — totalClaims, averageRiskScore, highRiskCount. `highRiskCount ≥ 2` = yüksək riskli.
- Chatbot:** sessiya tarixçəsi ilə çoxnövbəli söhbət (ChatMessage), sığorta köməkçisi personası.

Metod	Yol	Auth
GET	/api/v1/ai/fraud-checks/{claimId}	STAFF, ADMIN
GET	/api/v1/ai/risk-profile/{userId}	STAFF, ADMIN
POST	/api/v1/ai/chatbot	CUSTOMER
POST	/api/v1/ai/claims/{claimId}/analyze	STAFF, ADMIN

8. notification — Bildirişlər

Tək istiqamətli: **7 hadisənin hamısını qəbul edir**, heç nə yayımlamır. Hər hadisəni uyğun bildiriş tipinə çevirir. MVP-də real email/SMS yoxdur — `NotificationSender` sadəcə `log.info` ilə "göndərir" və SENT statusu qoyur.

Qəbul edilən hadisə	Yaranan bildiriş
user.registered	WELCOME (email mövcuddur)
policy.created	POLICY_PENDING
payment.completed	POLICY_ACTIVATED və ya PAYMENT_SUCCESS
payment.failed	PAYMENT_FAILED
claim.submitted	CLAIM_SUBMITTED
claim.decision	CLAIM_APPROVED / CLAIM_REJECTED
fraud.detected	FRAUD_ALERT (yalnız riskScore ≥ 70)

RetryScheduler: hər 5 dəqiqədə FAILED bildirişləri (retryCount < 3) yenidən göndərməyə çalışır.

Ümumi şablon: Bütün biznes servisləri eyni quruluşu paylaşır — `controller/ → service/(+impl) → repository/ → entity/ , dto/(request+response) , kafka/ , mapper/ (MapStruct), config/ , exception/ (GlobalExceptionHandler + XxxErrorEnum)`. Hər biri JWT-ni özü doğrulayır, hər biri öz `@PreAuthorize` rol yoxlamasını edir.